Graphic Era Hill University
DEHRADUN • BHIMTAL • HALDWANI

PROJECT AND TEAM INFORMATION

Project Title

TrustBox: Enterprise-Grade Digital Locker with Role-Based Access Control

Project GitHub URL

https://github.com/Anshika-111105/Full-stack-Auth-System

Student/Team Information

Team Name:	
Team member 1 (Team Lead) (Name, Student ID, Email, Picture): Name : Anshika Saklani University Roll no : 2318420 Student ID : 23012076 Email: anshikasaklani894@gmail.com	

Team member 2 (Name, Student ID, Email, Picture): Name : Ayush Chand University Roll no : 2318582 Student ID : 230112536 Email: ayushchand862@gmail.com	
Team member 3 (Name, Student ID, Email, Picture): Name : Akriti Rawat University Roll no : 2318263 Student ID : 230221541 Email: akritirawat12345@gmail.com	

Team member 4 (Name, Student ID, Email, Picture): Name : Rakhi Bisht University Roll no : 2319380 Student ID : 23012177 Email: rakhibisht24122004@gmail.com	

PROJECT PROGRESS DESCRIPTION

Project Abstract

(Brief restatement of your project's main goal. Max 300 words).

Traditional authentication systems focus solely on identity verification, leaving a critical gap in secure data management .TrustBox bridges this gap by combining enterprise-grade authentication with a fully integrated secure document management platform — evolving beyond a simple login system into a production-ready digital locker solution.

The system addresses real-world security challenges by implementing a multi-layered protection model. At its core, it retains the robust authentication foundation — stateless JWT-based session management, bcrypt password hashing, and Time-based One-Time Password (TOTP) two-factor authentication — and extends these protections to cover every document interaction within the platform.

Users can securely upload, view, search, categorize, and delete documents, while sensitive personal information is stored as server-side encrypted notes. A secure link-sharing mechanism enables controlled document sharing without exposing raw file access. All operations are governed by an expanded Role-Based Access Control (RBAC) system supporting three roles — User, Admin, and Verifier — ensuring granular permission enforcement across every API endpoint.

The backend is built on Node.js and Express.js, with MongoDB managing three core collections: documents, notes, and shared links. File uploads are handled via Multer with strict validation and size restrictions. The frontend dashboard provides real-time visibility into storage usage, document count, recent uploads, and active shared links — replacing all static mock data with live backend API calls.

SecureVault is designed to be scalable, maintainable, and aligned with enterprise security standards — making it a strong real-world alternative to platforms like DigiLocker. It demonstrates end-to-end full stack engineering, from authentication infrastructure to encrypted file management, suitable for both academic evaluation and professional portfolio presentation.

Updated Project Approach and Architecture

(Describe your current approach, including system design, communication protocols, libraries used, etc. Max 300 words).

TrustBox follows a modular, decoupled architecture based on the principle of defense-in-depth, ensuring multiple layers of security across both authentication and document management workflows.

The backend is built as a RESTful API using Node.js and Express.js, handling all client requests through clearly separated route modules — auth, documents, notes, and shared links. Authentication remains stateless and token-based using JWT, with TOTP-based Two-Factor Authentication via otplib ensuring verified identity before any sensitive operation is performed.

The expanded MongoDB database now manages three core collections alongside the existing users collection — documents, notes, and sharedlinks — modeled using Mongoose schemas with strict validation. Passwords continue to be hashed using bcrypt with salting. Encrypted notes are stored server-side to protect sensitive user data at rest.

File uploads are handled using Multer middleware, enforcing file type validation, size restrictions, and secure storage handling before any document is persisted. Secure sharing is implemented through tokenized links with expiry controls, preventing unauthorized raw file access.

Role-Based Access Control is expanded across three roles — User, Admin, and Verifier — enforced via Express middleware on every protected route. Rate limiting is applied globally to prevent brute-force and abuse attacks.

The frontend, built with React.js, communicates with the backend exclusively via HTTP/HTTPS API calls — completely replacing all previous static mock data with live fetch requests to backend endpoints. The dashboard renders real-time metrics including storage usage, document count, recent uploads, and active shared links.

Environment variables via dotenv manage all sensitive configuration. API testing and validation are performed using Postman across all new and existing endpoints.

Overall, TrustBox's architecture is scalable, security-first, and production-aligned.

Tasks Completed

(Describe the main tasks that have been assigned and completed. Max 250 words).

Task Completed	Team Member
Backend Development — JWT authentication, bcrypt password hashing, Express.js REST API setup	Anshika Saklani
TOTP-based Two-Factor Authentication (2FA) integration using otplib	Anshika Saklani
Role-Based Access Control (RBAC) middleware — User, Admin, Verifier roles	Anshika Saklani
Document Management APIs — Upload, View, Delete, Search endpoints	Anshika Saklani , Ayush Chand
File upload system using Multer — file type validation, size restrictions, storage handling	Ayush Chand , Rakhi Bisht
MongoDB schema design — documents, notes, sharedlinks, users collections	Rakhi Bisht
Encrypted Notes system — backend storage and retrieval of sensitive user data	Anshika Saklani
Secure link sharing — tokenized share links with expiry controls	Ayush Chand
Frontend UI Development — Dashboard, document listing, upload interface	Akriti Rawat
Frontend API Integration — replacing static mock data with live backend fetch calls	Akriti Rawat , Ayush Chand
API Testing & Validation using Postman across all endpoints	Akriti Rawat, Ayush Chand
Environment configuration using dotenv	Rakhi Bisht

Challenges/Roadblocks

(Describe the challenges that you have faced. Max 300 words).

1. Email-Based OTP Reliability During the initial 2FA implementation, OTP delivery via email faced significant issues including delayed delivery, spam filtering, and SMTP service dependency. OTPs frequently expired before reaching users, degrading both security and experience. This was resolved by migrating to TOTP-based authentication using otplib, generating real-time codes directly on the user's device via Google Authenticator — eliminating external service dependency entirely.

2. Secure File Upload Handling Implementing Multer for file uploads introduced challenges around file type spoofing, oversized uploads, and unsafe storage paths. Strict MIME type validation, file size restrictions, and sanitized storage naming conventions were enforced to prevent malicious file injection and storage abuse.

3. Secure Link Sharing with Expiry Control Generating tokenized share links that expire correctly required careful handling of token generation, storage in the sharedlinks collection, and real-time expiry validation on every access request. Edge cases such as already-expired links and revoked access required additional middleware logic.

4. Frontend Migration from Static to Live Data The existing frontend relied entirely on hardcoded mock arrays for document display. Replacing these with asynchronous fetch calls to backend APIs introduced challenges around loading states, error handling, and maintaining UI consistency during data fetching — all of which were resolved through structured API response handling.

5. RBAC Expansion Across New Routes Extending Role-Based Access Control to cover three roles — User, Admin, and Verifier — across all new document, notes, and sharing endpoints required careful middleware chaining to avoid permission gaps or over-restriction on nested routes.

6. Rate Limiting Implementation Applying global rate limiting without disrupting legitimate high-frequency operations such as dashboard refresh required fine-tuned configuration of rate limit thresholds per route category.

Tasks Pending

(Describe any task that you still need to complete. Max 250 words).

Tasks Pending	
Cloud Storage Integration	Migrate file storage from local to AWS S3 or Cloudinary for scalable, distributed document hosting
Document Versioning	Maintain version history for uploaded documents — allowing users to restore previous versions
Full-Text Document Search	Implement Elasticsearch or MongoDB Atlas Search for fast content-level document searching
Document Preview	In-browser PDF and image preview without requiring download

Project Outcome/Deliverables

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

Here's the updated Project Outcome/Deliverables section:

TrustBox delivers a production-ready, enterprise-grade secure document management platform built on a robust full stack authentication foundation. The key deliverables are as follows:

Authentication & Security

- Stateless JWT-based authentication system with bcrypt password hashing
- TOTP-based Two-Factor Authentication integrated via otplib
- Expanded Role-Based Access Control — User, Admin, and Verifier roles
- Global rate limiting and input sanitization across all endpoints

Document Management

- Secure document upload, view, search, categorize, and delete functionality
- Multer-based file upload system with strict type validation and size restrictions
- Tokenized secure link sharing with expiry controls
- Server-side encrypted personal notes storage

Database & Backend

- MongoDB collections — users, documents, notes, sharedlinks — modeled via Mongoose
- Fully tested REST APIs validated through Postman
- Environment-based configuration using dotenv

Frontend & Dashboard

- React.js dashboard displaying real-time storage usage, document count, recent uploads, and active shared links
- Complete replacement of static mock data with live backend API integration

Documentation & Deployment

- Comprehensive API documentation
- Deployment-ready environment configuration

Overall, TrustBox demonstrates end-to-end full stack engineering suitable for real-world enterprise deployment.

Progress Overview

(Summarize how much of the project is done, and what's left. Max 200 words.)

TrustBox is approximately **75–85% complete**, with all core backend systems fully functional and frontend integration actively in progress.

Completed:

- JWT authentication, bcrypt password hashing, and TOTP-based 2FA are fully implemented and tested
- Expanded RBAC middleware covering User, Admin, and Verifier roles is operational
- All document management APIs — upload, view, delete, search — are built and Postman validated
- MongoDB schema design across all four collections is complete
- Encrypted notes system and tokenized secure link sharing are functional
- Frontend dashboard structure is built with live backend API integration replacing all static mock data

In Progress:

- Admin dashboard — user management interface and file oversight panel
- Frontend UI enhancements — responsiveness, loading states, and error boundary handling
- API documentation and schema documentation

Pending:

- Final security audit covering token misuse, rate limit fine-tuning, and input sanitization
- Edge case testing — expired tokens, invalid TOTP, revoked share links
- Production deployment and environment configuration finalization

The project is on track with no major blockers. Remaining work is focused on polishing, documentation, and deployment rather than core feature development.

Testing and Validation Status

(Provide information about any tests conducted)

Test Type	Status (Pass/Fail)	Notes
User Registration & Login API	Pass	Successfully tested with valid and invalid inputs, duplicate email handling verified
JWT Authentication & Authorization	Pass	Token generation, expiry, and protected route access working correctly
Two-Factor Authentication (TOTP)	Pass	Codes generated and validated correctly via Google Authenticator

Document Upload API (Multer)	Pass	File type validation, size restrictions, and storage handling verified
RBAC — User, Admin, Verifier Roles	Pass	Role-based route restrictions enforced correctly across all protected endpoints
Edge Cases — Expired Tokens & Invalid TOTP	Partial Pass	Core cases handled, minor edge cases under final testing

Deliverables Progress

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

The core backend infrastructure of TrustBox is fully completed. This includes the Node.js and Express.js REST API, stateless JWT-based authentication, bcrypt password hashing and salting, and TOTP-based Two-Factor Authentication integrated via otplib. The expanded Role-Based Access Control system covering User, Admin, and Verifier roles is operational, with middleware enforced across all protected routes. All document management APIs — upload, view, delete, and search — are built and validated. The encrypted notes system and tokenized secure link sharing with expiry controls are fully functional. MongoDB schema design across all four collections — users, documents, notes, and sharedlinks — is complete. Rate limiting, input sanitization, and full API testing via Postman are also completed.

Currently in progress are the React.js frontend dashboard and admin interface, where live backend API integration is functional but UI responsiveness, loading states, and error boundary handling are being refined. The admin dashboard's user management and file oversight panel is actively under development. API and schema documentation is being written in parallel, and edge case testing covering expired tokens, invalid TOTP codes, and revoked share links is in its final stages. The final security audit covering token misuse prevention and rate limit fine-tuning is also ongoing.

Production deployment and environment configuration remain the only fully pending deliverable, planned as the final step once all testing and documentation are complete.

Overall, 10 out of 16 deliverables are fully completed, with the remaining work concentrated on frontend polish, documentation, and deployment.